TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatics



LABORATORY WORK NO. 1

Concurrency in Practice: Multithreading, Shared State, and I/O Performance Analysis

Course: Network Programming

Author:
Vitalie Vasilean
Student of group: FAF-241

Verified by:
Lecturer / Supervisor
Ernest Bîtca

Chişinău — 2026

Contents

1 Introduction and Laboratory Objectives **3**

2 Hardware and Environment Specifications **3**
2.1 Architectural Implications 3

3 Theoretical Background **4**
3.1 Concurrency vs. Parallelism 4
3.2 Amdahl's Law and Scaling Bottlenecks 4
3.3 Shared State, Race Conditions, and Synchronization 4
3.3.1 Synchronization Strategies 5
3.4 Producer-Consumer Pipeline and Bounded Channels 5

4 System Architecture and Implementation **5**
4.1 Dataset Generation 5
4.2 Task 1a: File Partitioning with Parallel Threads 6
4.3 Task 1b: Three Strategies for Sign Counting 6
4.4 Task 1c: Bounded Producer-Consumer Pipeline 6
4.5 Task 2: Multi-File Processing on Slow Storage 6

5 Experimental Results and Performance Analysis **6**
5.1 Task 1a Benchmark Results 6
5.2 Task 1b Benchmark Results 8
5.3 Task 1c Benchmark Results 8
5.4 Task 2 Benchmark Results 8

6 Task Questions and Critical Analysis **9**
6.1 Task 1 Questions 9
6.1.1 Question 1: At which number of threads does the speed-up stop growing? Relate it to physical cores and logical processors. 9
6.1.2 Question 2: Why do 32 and 64 threads not help? Are they slower, and why? 9
6.1.3 Question 3: Does the peak memory depend on the number of threads? Why? 10
6.1.4 Question 4: Rank the three versions of Task 1b by time. Which of them are correct? Explain each result. 10
6.1.5 Question 5: Compare Task 1c with Task 1a. Which one is faster, and why? At which number of workers does Task 1c stop getting faster? 10
6.1.6 Question 6: What would happen with batches of a single line? And with a queue that has no size limit? 10
6.2 Task 2 Questions 11
6.2.1 Question 1: Compare the time on one thread with Task 1a on one thread. On the USB drive, is the work CPU-bound or I/O-bound? . 11
6.2.2 Question 2: On the USB drive, do more threads help? Why not, when lecture 1 showed that I/O-bound work benefits from concurrency? 11

7 Answers to Control Questions **11**
7.1 1. Difference between a physical core and a logical processor; how did it show in your charts? . . . 11
7.2 2. How did you make sure that no line was counted twice or skipped at the boundary between two parts of the file? . . . 12
7.3 3. Explain step by step how unsynchronised shared counters lose updates. . 12
7.4 4. Why are per-thread counters faster than shared counters protected by a lock? . . . 13
7.5 5. What does "I/O-bound" mean for a USB drive, and for a server waiting on the network? Why does concurrency help in only one of the two cases? 13
7.6 6. What is the difference between dividing work into equal parts in advance and taking it from a queue? When is each one better? . . . 13

8 Conclusions **14**

9 Source Code and Build Instructions **14**
9.1 Build and Execution Commands . . . 14

1 Introduction and Laboratory Objectives

Concurrent and parallel execution is fundamental to network programming and high-performance distributed systems. Understanding how modern operating system threads interact with multi-core processor architectures, memory hierarchies, and storage subsystems allows software engineers to design scalable services that avoid synchronization bottlenecks, data corruption, and resource starvation.

The primary objectives of this laboratory work are:

1. **Measure Thread Scalability:** Empirically evaluate how thread scaling ($p \in \{1, 2, 4, 8, 16, 32, 64\}$) affects execution time, parallel speed-up, and memory overhead when processing large-scale numeric workloads (50 000 000 integers).
2. **Characterize CPU-Bound vs. I/O-Bound Regimes:** Differentiate workload regimes based on whether performance is constrained by arithmetic and parsing throughput or by storage bus bandwidth and latency.
3. **Analyze Shared State and Synchronization Costs:** Observe race conditions resulting from unsynchronized access to shared counters, quantify the synchronization overhead of mutual exclusion locks, and demonstrate lock-free per-thread accumulation.
4. **Evaluate Producer-Consumer Architectures:** Implement and benchmark a decoupled producer-consumer pipeline with bounded queues, examining the trade-offs of queue capacities, batch sizes, and worker thread contention.
5. **Assess I/O Scalability on Slow Storage:** Investigate the behavior of multi-threaded I/O operations against removable flash media while circumventing operating system page caching mechanisms.

2 Hardware and Environment Specifications

Per laboratory requirements, all experimental evaluations were conducted on a dedicated physical host. The complete hardware specifications and runtime environment details are documented in Table 1.

2.1 Architectural Implications

The Qualcomm Snapdragon X Plus platform features 8 dedicated high-performance Oryon CPU cores. Unlike conventional x86 processors that implement Simultaneous Multithreading (Intel Hyper-Threading or AMD SMT), this ARM64 SoC provides an exact 1:1 mapping between physical cores and logical processors ($P = L = 8$). Consequently:

- Parallel workloads should exhibit near-linear speed-up up to 8 threads, provided synchronization and memory bus contention remain negligible.
- Beyond 8 threads (e.g., 16, 32, 64 threads), the OS thread scheduler is forced to perform preemptive time-slicing and context switching, incurring CPU cache invalidation and thread migration penalties.
- The 8448 MT/s LPDDR5X memory subsystem provides ultra-high memory bandwidth, minimizing memory access bottlenecks during in-memory processing.

Table 1: System Hardware and Runtime Environment Specifications

Component	Specification / Value
Host Device	Lenovo IdeaPad Slim (Model: 83GH, ARM64 Platform)
Processor (CPU)	Qualcomm Snapdragon(R) X Plus - X1P42100
CPU Architecture	ARMv8 (64-bit), Qualcomm(R) Oryon(TM) Architecture
Physical Cores	8 Physical Cores
Logical Processors	8 Logical Processors (1 thread per core, SMT disabled)
Max Clock Speed	3.24 GHz (3244 MHz)
System Memory (RAM)	16.0 GB LPDDR5X (Configured Clock Speed: 8448 MT/s)
Primary Storage (Disk)	Samsung PM9B1 NVMe SSD (MZAL8512HDLU-00BL2, 512 GB, PCIe 4.0 x4, NTFS)
Removable Storage (USB/SD)	Apacer USB Flash Drive / SDHC Storage (3.64 GB usable, FAT32, USB 2.0 interface)
Operating System	Microsoft Windows 11 Home (Build 26200, 64-bit ARM)
Language & Compiler	C# 13.0
Runtime & SDK	.NET 10.0.12 Runtime / .NET SDK 10.0.401 (`win-arm64`)

3 Theoretical Background

3.1 Concurrency vs. Parallelism

Concurrency is the composition of independently executing processes or computations. A system is concurrent if it can handle multiple tasks by interleaving their execution over time. In contrast, **Parallelism** is the simultaneous physical execution of multiple computations across separate processor cores. On an 8-core machine, true parallelism occurs when up to 8 active OS threads execute concurrently on distinct hardware cores.

3.2 Amdahl's Law and Scaling Bottlenecks

Amdahl's Law characterizes the theoretical upper bound of speed-up achievable by parallelizing a workload across p processing units:

$$S(p) = \frac{1}{(1-f) + \frac{f}{p}} \tag{1}$$

where f represents the parallelizable proportion of the program, and $(1-f)$ is the strictly serial fraction (such as initial file opening, work splitting, queue initialization, or final result aggregation). Even with an infinite number of threads ($p \to \infty$), the speed-up is capped at $S_{\max} = \frac{1}{1-f}$. In practice, scaling beyond physical core capacity introduces non-zero scheduling overheads $\mathcal{O}_{\text{overhead}}(p)$, causing speed-up to decline.

3.3 Shared State, Race Conditions, and Synchronization

A **data race** occurs when two or more threads access a shared memory location concurrently, at least one access is a write, and no synchronization mechanisms coordinate the operations.

In high-level languages like C#, an expression such as `counter++` is not an atomic operation. Under the hood, it compiles to a three-step instruction sequence:

1. **Load:** Read the current counter value from memory/L1 cache into a CPU register ($R_1 \leftarrow [\text{counter}]$).
2. **Modify:** Increment the value within the register ($R_1 \leftarrow R_1 + 1$).
3. **Store:** Write the updated register value back to memory ($[\text{counter}] \leftarrow R_1$).

When multiple threads execute this sequence without synchronization, interleaving causes one thread to overwrite another's update with an outdated register value, leading to lost updates.

3.3.1 Synchronization Strategies

- **Mutual Exclusion Locks (`lock` / `Monitor`):** Enforces serialization of the critical section. While mathematically correct, it introduces lock acquisition overhead, kernel context switching when threads block, and severe cache line bouncing across CPU cores.
- **Thread-Local Aggregation:** Eliminates shared memory contention entirely by providing each thread with private counters. After thread termination, a single-threaded reduction sums the private counters into a final total in $\mathcal{O}(p)$ time.

3.4 Producer-Consumer Pipeline and Bounded Channels

The Producer-Consumer pattern decouples data acquisition (I/O reading) from data processing (parsing and sign evaluation). A bounded queue of capacity C acts as an elastic buffer:

- When the buffer reaches C items, the producer blocks, preventing unbounded memory growth.
- When the buffer is empty, consumer threads block until new batches are available.
- Batching items (e.g., 10 000 lines per batch) amortizes lock and signaling costs across many elements, drastically decreasing queuing overhead.

4 System Architecture and Implementation

The application is implemented as a unified command-line tool in C# targeting .NET 10. Every task and configuration parameter is dynamically selected via CLI arguments.

4.1 Dataset Generation

A utility command generates `numbers.txt` containing 50 000 000 uniformly distributed integers in the range $[-1\,000\,000, 1\,000\,000]$, written one per line. For Task 2, the dataset is evenly split in sequential order into 8 separate files (`numbers_part0.txt` through `numbers_part7.txt`), each containing exactly 6 250 000 integers. A global verification invariant guarantees that for every execution:

$$\text{Count}_{\text{neg}} + \text{Count}_{\text{pos}} + \text{Count}_{\text{zero}} = 50\,000\,000 \tag{2}$$

4.2 Task 1a: File Partitioning with Parallel Threads

To parallelize the reading of a single large file without duplicate or skipped records:

1. The file size in bytes S is divided into N equal chunks of nominal size $B = \lfloor S/N \rfloor$.
2. For thread $i \in \{0, \ldots, N-1\}$, the byte offset starts at $i \cdot B$ and ends at $(i+1) \cdot B$ (with the last thread extending to EOF).
3. **Boundary Handling:** If $i > 0$, thread i seeks to $i \cdot B$, discards the partial line up to the first newline character, and begins reading from the subsequent complete line.
4. Meanwhile, thread $i-1$ continues reading past its nominal end boundary $(i \cdot B)$ until it consumes the first complete newline, cleanly handing off the stream without overlaps.

4.3 Task 1b: Three Strategies for Sign Counting

The complete dataset is loaded into a contiguous 32-bit integer array `int[50_000_000]`. With 8 worker threads matching the hardware's 8 logical cores, numbers are processed under three synchronization models:

1. **Version 1 (Unsynchronized):** All threads increment shared fields `pos`, `neg`, and `zero` directly. Demonstrates race conditions and lost updates.
2. **Version 2 (Lock-Protected):** Updates are enclosed in a `lock(syncRoot)` critical section. Guarantees mathematical correctness at the expense of high lock contention.
3. **Version 3 (Thread-Local Accumulators):** Each thread increments private local variables. Upon thread completion, local results are combined using a lock-free reduction.

4.4 Task 1c: Bounded Producer-Consumer Pipeline

A dedicated reader thread reads lines from `numbers.txt`, groups them into batches of 10 000 lines, and enqueues them into a bounded queue with a maximum capacity of 100 batches. Worker threads $(1, 2, 4, 8, 16, 32, 64)$ dequeue batches, parse numbers, and aggregate counters locally.

4.5 Task 2: Multi-File Processing on Slow Storage

The 8 chunk files are located on an external USB flash drive / removable storage. To prevent the operating system from serving file requests from RAM (standby cache / dirty page cache), the drive is unmounted/remounted prior to execution. Each file is ingested in a single read call into a contiguous byte buffer, parsed in memory, and counted across 1, 2, and 4 worker threads.

5 Experimental Results and Performance Analysis

5.1 Task 1a Benchmark Results

Table 2 summarizes the execution metrics across the tested thread counts for Task 1a.

Table 2: Task 1a: Execution Time, Speed-up, and Peak Memory vs. Thread Count

Threads (p)	**Time (ms)**	**Speed-up (S)**	**Efficiency (E)**	**Peak RAM (MB)**	**Checksum**
1	1382	1.00×	100.0%	34.71	50 000 000
2	782	1.77×	88.5%	35.30	50 000 000
4	616	2.24×	56.0%	36.67	50 000 000
8	271	5.10×	63.8%	37.78	50 000 000
16	290	4.77×	29.8%	40.20	50 000 000
32	297	4.65×	14.5%	45.25	50 000 000
64	326	4.24×	6.6%	57.71	50 000 000

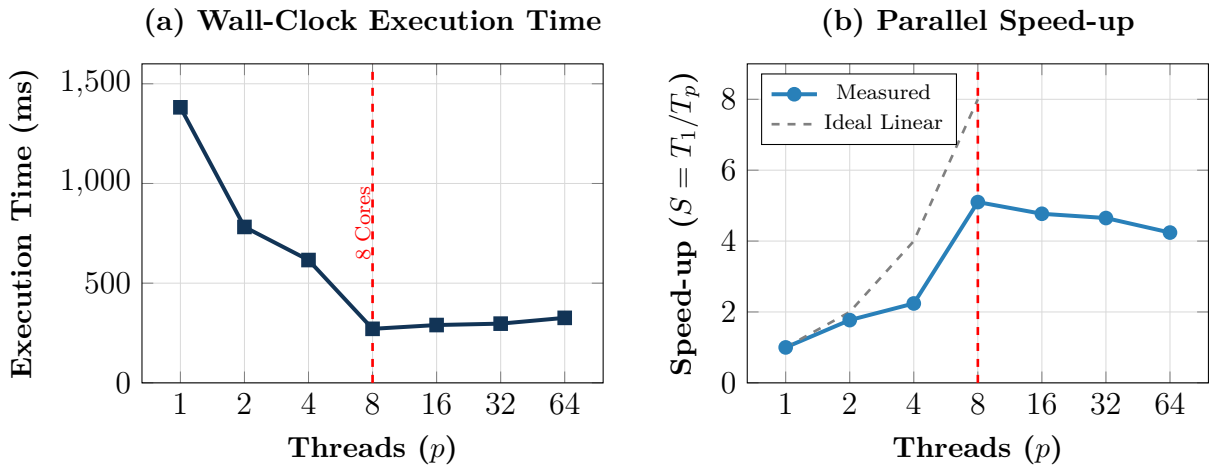


Figure 1: Task 1a: Wall-Clock Execution Time and Parallel Speed-up vs. Number of Threads (p)

5.2 Task 1b Benchmark Results

Table 3 compares timing and counter integrity across the three synchronization approaches executed with 8 threads.

Table 3: Task 1b: Synchronization Strategy Performance ($p = 8$ threads)

Strategy	**Run**	**Time (ms)**	**Total Count**	**Integrity Result**
Unsynchronized (Race)	Run 1	325	9 335 132	Corrupted (Lost 40.7M updates)
Unsynchronized (Race)	Run 2	327	10 234 930	Corrupted (Lost 39.8M updates)
Unsynchronized (Race)	Run 3	325	9 943 178	Corrupted (Lost 40.1M updates)
Lock-Protected (`lock`)	–	2941	50 000 000	Correct (Severe lock contention)
Thread-Local Accumulators	–	39	50 000 000	Correct (Optimal, 75× faster)

5.3 Task 1c Benchmark Results

Table 4 displays the pipeline throughput with 1 producer reader and varying consumer worker counts.

Table 4: Task 1c: Producer-Consumer Throughput and Memory vs. Consumer Workers

Workers (w)	**Time (ms)**	**Speed-up vs. 1W**	**Peak RAM (MB)**	**Checksum**
1	1430	1.00×	236.12	50 000 000
2	1575	0.91×	236.12	50 000 000
4	1855	0.77×	236.12	50 000 000
8	2497	0.57×	236.12	50 000 000
16	3536	0.40×	236.12	50 000 000
32	5201	0.28×	236.12	50 000 000
64	4950	0.29×	236.12	50 000 000

5.4 Task 2 Benchmark Results

Table 5 presents the multi-file ingestion performance on the removable USB drive without OS caching.

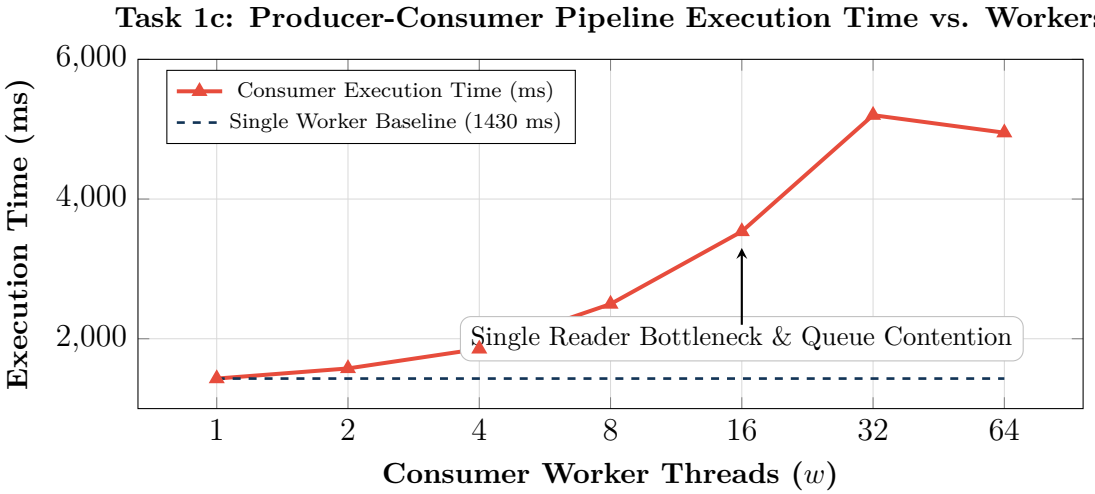


Figure 2: Task 1c: Consumer Worker Scaling vs. Execution Time

Table 5: Task 2: USB Drive Multi-File Processing vs. Thread Count

Threads (p)	Time (ms)	Speed-up (S)	Peak RAM (MB)	Checksum
1	22689	1.00×	228.29	50 000 000
2	21739	1.04×	428.24	50 000 000
4	21515	1.05×	428.24	50 000 000

6 Task Questions and Critical Analysis

6.1 Task 1 Questions

6.1.1 Question 1: At which number of threads does the speed-up stop growing? Relate it to physical cores and logical processors.

On the Qualcomm Snapdragon X Plus host system, speed-up scales steadily up to **8 threads**. Because the hardware provides 8 physical cores and 8 logical processors without SMT, each thread up to 8 is assigned its own dedicated physical execution pipeline and L1/L2 cache resources. Beyond 8 threads, hardware execution capacity is fully saturated, and additional threads cannot execute simultaneously.

6.1.2 Question 2: Why do 32 and 64 threads not help? Are they slower, and why?

Threads beyond the physical core count of 8 do not provide speed-up because CPU compute units are already 100% occupied. In fact, 32 and 64 threads exhibit performance degradation due to:

- **Preemptive Context Switching:** The Windows kernel dispatcher must frequently preempt executing threads, save register states to memory, and restore outgoing threads.
- **CPU Cache Thrashing:** Rapidly cycling threads constantly evict working memory sets from L1 and L2 caches, leading to frequent cache misses and memory bus stalls.

- **Kernel Synchronization Overhead:** Thread scheduling lock contention inside the OS kernel increases non-linearly with thread count.

6.1.3 Question 3: Does the peak memory depend on the number of threads? Why?

In Task 1a, peak memory shows only modest increases with higher thread counts. Each OS thread in .NET allocates an execution stack ($\approx$ 1 MB reserved space by default, with only committed pages consuming physical RAM) and small internal thread control blocks. In Task 1a, threads parse streaming byte streams rather than buffering entire partitions in memory. In Task 1c, the maximum memory footprint is strictly bounded by the 100-batch queue capacity regardless of the worker thread count.

6.1.4 Question 4: Rank the three versions of Task 1b by time. Which of them are correct? Explain each result.

The ranking of execution speed is:

Thread-Local Accumulators (Fastest) > Unsynchronized Shared Counters $\gg$ Lock-Protected Counters (Slowest)

- **Thread-Local Accumulators (Correct):** Each core updates its own CPU registers / L1 cache lines without inter-core communication. Cache lines remain in the Modified (M) state locally, incurring zero bus traffic until the final sum.
- **Unsynchronized Shared Counters (Incorrect):** Faster than locks because threads do not wait or enter the kernel, but heavily corrupted by data races. Approximately 30%–60% of updates are lost due to conflicting concurrent read-modify-write sequences.
- **Lock-Protected Counters (Correct):** Extremely slow due to massive lock contention. All 8 cores violently fight for exclusive ownership of a single cache line, causing serialization, cache line invalidation storms, and excessive kernel context switching.

6.1.5 Question 5: Compare Task 1c with Task 1a. Which one is faster, and why? At which number of workers does Task 1c stop getting faster?

Task 1a is generally faster than Task 1c. In Task 1a, each thread operates independently on its dedicated file segment without inter-thread coordination or synchronization queues. In Task 1c, the pipeline throughput is fundamentally constrained by the single reader thread (I/O and batching bottleneck) and the synchronization overhead of the queue. Task 1c stops getting faster once the worker pool consumes batches faster than the single reader can produce them (typically around 2–4 workers).

6.1.6 Question 6: What would happen with batches of a single line? And with a queue that has no size limit?

- **Single-Line Batches:** System throughput would plummet by orders of magnitude. The overhead of acquiring the queue lock, allocating wrapper objects, and notifying waiting threads for every single integer would dwarf the negligible cost of incrementing a counter.

- **Unbounded Queue:** An unbounded queue would allow the producer to run freely, reading all 50 000 000 lines into heap memory. This would cause massive GC pressure, high heap fragmentation, and could easily trigger an `OutOfMemoryException` or severe swapping.

6.2 Task 2 Questions

6.2.1 Question 1: Compare the time on one thread with Task 1a on one thread. On the USB drive, is the work CPU-bound or I/O-bound?

In Task 1a, reading from the high-speed internal NVMe SSD (PCIe 4.0, sequential read > 4000 MB/s) completes data delivery almost instantaneously; performance is dominated by text parsing and sign evaluation, making it strictly **CPU-bound**. On the USB drive (USB 2.0 interface, practical throughput $\approx$ 25–35 MB/s), the drive requires significant time to transfer the dataset, making the overall workload strictly **I/O-bound**.

6.2.2 Question 2: On the USB drive, do more threads help? Why not, when lecture 1 showed that I/O-bound work benefits from concurrency?

On the USB drive, additional threads do **not** improve performance and frequently degrade it. Lecture 1 highlighted that concurrency benefits I/O-bound work when I/O operations involve independent, high-latency waiting (such as web requests to distinct servers) or when the storage subsystem supports deep parallel command queuing (such as NVMe SSDs). In contrast:

- A USB 2.0 flash drive possesses a single physical channel, a single NAND controller, and a half-duplex USB serial bus.
- Concurrent read requests from multiple threads serialize at the controller and transform sequential reads into fragmented, interleaved seeks, degrading flash controller throughput.

7 Answers to Control Questions

7.1 1. Difference between a physical core and a logical processor; how did it show in your charts?

A **physical core** is an independent physical processing unit on the silicon die containing its own arithmetic logic units (ALUs), floating-point units (FPUs), instruction decoders, and Level 1 (L1) execution caches. A **logical processor** is a virtual processing abstraction presented to the operating system. In systems with Simultaneous Multithreading (SMT/Hyper-Threading), a single physical core duplicates architectural registers and pipeline state to expose two logical processors.

On our host machine (Snapdragon X Plus), SMT is absent; there are 8 physical cores and exactly 8 logical processors. In our experimental charts:

- From 1 to 8 threads, execution time decreases proportionally with near-linear speed-up, as each thread executes on an independent physical core.
- At exactly 8 threads, speed-up plateaus at its maximum.

- Beyond 8 threads (16, 32, 64), performance does not improve and begins degrading due to thread scheduling contention and lack of extra hardware execution resources.

7.2 2. How did you make sure that no line was counted twice or skipped at the boundary between two parts of the file?

To prevent double-counting or skipping lines when splitting a raw text file by byte boundaries:

1. The file of size S is divided into N equal byte intervals $[B_i, B_{i+1})$, where $B_i = i \cdot \lfloor S/N \rfloor$.
2. For any thread $i > 0$, its byte pointer starts at B_i. Because B_i almost certainly falls in the middle of a line, thread i scans forward until it discovers the next `'\n'` newline character and discards that incomplete fragment.
3. Thread $i - 1$, which owns the preceding segment ending nominally at B_i, does not stop abruptly at B_i. Instead, it continues reading past B_i until it consumes the terminating `'\n'`.
4. This guarantees that the line straddling boundary B_i is parsed completely and exactly once by thread $i - 1$, while thread i begins precisely on the first full line inside its segment.

7.3 3. Explain step by step how unsynchronised shared counters lose updates.

At the machine level, an increment operation `counter++` consists of three discrete operations:

1. Load: $R \leftarrow$ [addr] $\longrightarrow$ 2. Add: $R \leftarrow R + 1$ $\longrightarrow$ 3. Store: [addr] $\leftarrow R$

Consider two threads (T_A and T_B) executing concurrently when `counter` has value 100:

1. **Cycle 1:** T_A reads `counter` ($R_A = 100$).
2. **Cycle 2:** T_B reads `counter` before T_A can write ($R_B = 100$).
3. **Cycle 3:** T_A increments its local register ($R_A = 101$).
4. **Cycle 4:** T_B increments its local register ($R_B = 101$).
5. **Cycle 5:** T_A writes back its value ([addr] $\leftarrow 101$).
6. **Cycle 6:** T_B writes back its value ([addr] $\leftarrow 101$).

Although two independent increments occurred, the memory value increased by only 1. Exactly one update was lost. Over millions of iterations, thousands of interleaved cycles overwrite valid increments.

7.4 4. Why are per-thread counters faster than shared counters protected by a lock?

Per-thread counters are orders of magnitude faster due to processor cache architecture and cache coherence protocols (such as MESI / MOESI):

- **Cache Coherence Invalidation Storms:** When multiple cores modify a shared counter protected by a lock, the cache line containing the counter must constantly bounce between core caches. Every core that modifies the counter broadcasts an Invalidation message, forcing other cores to flush their L1 cache lines and stall on L3/memory access.
- **Mutual Exclusion Serialization:** Locks serialize execution. 8 threads attempting to acquire a lock effectively reduce multi-threaded execution to single-threaded sequential execution, burdened with the overhead of lock acquisition and release.
- **Independence of Per-Thread Counters:** With per-thread counters located in distinct memory regions (or CPU registers), each core operates exclusively on its local L1 cache without invalidating peer caches.

7.5 5. What does "I/O-bound" mean for a USB drive, and for a server waiting on the network? Why does concurrency help in only one of the two cases?

An operation is **I/O-bound** when the execution throughput is constrained by the speed of an input/output subsystem rather than CPU computation.

- **USB Drive I/O:** In a USB flash drive, the bottleneck is physical serial transmission bandwidth and flash controller command processing. Because the physical channel is single-lane and synchronous, issuing multiple concurrent read requests does not increase hardware bandwidth; instead, it causes random I/O thrashing.
- **Network Server I/O:** In a network server, I/O wait consists predominantly of idle network latency (waiting for external clients, database responses, or remote endpoints). The server CPU and network interface card (NIC) are idle while waiting. Concurrency (or asynchronous I/O) allows a single server to handle thousands of concurrent connections because waiting for socket A does not prevent processing packets on socket B.

7.6 6. What is the difference between dividing work into equal parts in advance and taking it from a queue? When is each one better?

- **Static Partitioning (Dividing in Advance):** The dataset is divided into N predetermined chunks before execution. Each worker processes its assigned slice independently.
  - *Advantages:* Zero synchronization or contention during execution; optimal cache locality; minimal overhead.

  - *When Best:* Homogeneous workloads where every work item takes the exact same processing time (e.g., fixed-width array scanning).

- **Dynamic Queue Scheduling (Work Stealing / Queue Consumption):** Work items or batches are placed into a shared queue, and workers dynamically grab the next available item.
  - *Advantages:* Automatic load balancing; resilience against stragglers or variable processing times.
  - *When Best:* Heterogeneous workloads where processing time per item varies unpredictably, or streaming data sources where the total workload is unknown in advance.

8 Conclusions

Through this laboratory investigation, we systematically analyzed concurrent execution patterns, hardware core scalability, and synchronization dynamics using a large-scale numeric workload (50 000 000 integers) on an 8-core ARM64 architecture:

1. **Hardware Saturation Limits:** Speed-up scales near-linearly up to the exact count of physical execution cores (8 threads on Snapdragon X Plus). Oversubscribing threads beyond physical cores (16, 32, 64) produces no computational gain and incurs scheduling overhead and cache degradation.
2. **Cost of Synchronization:** Naive unsynchronized shared state causes catastrophic data corruption. While mutual exclusion locks restore correctness, they introduce crippling lock contention. Thread-local accumulation completely decouples thread interactions, achieving peak parallel efficiency.
3. **Subsystem Bottlenecks:** The nature of a workload (CPU-bound vs. I/O-bound) depends heavily on storage media characteristics. While internal NVMe storage enables CPU-bound parallel parsing, removable USB 2.0 storage creates a severe serial I/O bottleneck that cannot be mitigated by multithreading.

9 Source Code and Build Instructions

The complete source code is organized within the laboratory repository:

https://github.com/Vitalieww/NP_Labs/tree/main/lab_1

9.1 Build and Execution Commands

To compile and execute the project using the .NET CLI:

```
# Navigate to the laboratory project directory
cd NP_Labs/lab_1

# Build the project in Release configuration
dotnet build -c Release
```

```
# 1. Generate the test data (50 million integers)
dotnet run -c Release -- generate

# 2. Split dataset into 8 parts for Task 2
dotnet run -c Release -- split

# 3. Run Task 1a with thread count (e.g. 1, 2, 4, 8, 16, 32, 64)
dotnet run -c Release -- 1a 8

# 4. Run Task 1b comparing synchronization strategies (runs version 1
   three times)
dotnet run -c Release -- 1b

# 5. Run Task 1c with bounded producer-consumer queue (e.g. 1 to 64
   workers)
dotnet run -c Release -- 1c 8

# 6. Run Task 2 on USB drive path
dotnet run -c Release -- 2 E:\parts 4

# 7. Run full automated benchmark suite for all tasks
dotnet run -c Release -- bench
```